SLP-NLP Lab 2

Επεξεργασία Φωνής και Φυσικής Γλώσσας
Εαρινό Εξάμηνο 2025-26

Μαντζώρος Γεράσιμος, Παπαδημητρίου Νίκη

Απρίλιος 2026

Περιεχόμενα

Μέρος Α΄: Προπαρασκευή

Για τα ζητούμενα της προπαρασκευής χρησιμοποιήσαμε το **Sentence Polarity Dataset (MR)**.

Ζητούμενο 1 – Κωδικοποίηση Επισημειώσεων

Λιστινγ 1: Έ ύμ 1 (main.py:ΕΧ1)

```
% ΤΟΔΟ:
                                10                    λαβελς
```

Ζητούμενο 2 – Λεκτική Ανάλυση (Tokenization)

Λιστινγ 2: Έξοδος Ζητούμενου 2 (δαταλοαδινγ.πψ:ΕΞ2)

```
% ΤΟΔΟ:                                                                10
                  τοκενιζεδ
```

Ζητούμενο 3 – Κωδικοποίηση Παραδειγμάτων (Λέξεων)

Λιστινγ 3: Έξοδος Ζητούμενου 3 (δαταλοαδινγ.πψ:ΕΞ3)

```
% ΤΟΔΟ:                                        5
                                             ΣεντενςεΔαταςετ
```

Ζητούμενο 4 – Embedding Layer

Γιατί αρχικοποιούμε το embedding layer με τα προ-εκπαιδευμένα word embeddings·

Γιατί κρατάμε παγωμένα τα βάρη του embedding layer κατά την εκπαίδευση·

Ζητούμενο 5 – Output Layer(s)

Γιατί βάζουμε μη γραμμική συνάρτηση ενεργοποίησης στο προτελευταίο layer·

Τι διαφορά θα είχε αν είχαμε 2 ή περισσότερους γραμμικούς μετασχηματισμούς στη σειρά·

Ζητούμενο 6 – Forward Pass

Διαισθητική ερμηνεία της αναπαράστασης κέντρο-βάρους:

Πιθανές αδυναμίες της προσέγγισης για αναπαράσταση κειμένων:

Ζητούμενο 7 – DataLoaders

Συνέπειες μικρών και μεγάλων mini-batches στην εκπαίδευση:

Γιατί ανακατεύουμε τη σειρά των mini-batches σε κάθε εποχή·

Ζητούμενο 8 – Βελτιστοποίηση

Κριτήριο (Loss):

Optimizer:

Υπερπαράμετροι:

- Learning rate:
- Batch size:
- Εποχές:
- Άλλες:

Ζητούμενο 9 – Εκπαίδευση

Λιστινγ 4: Σύνοψη εκπαίδευσης

```
% TODO:                                                    (   .
      . loss                        )
```

Ζητούμενο 10 – Αξιολόγηση Baseline Μοντέλου

Πίνακας 1: Αποτελέσματα Baseline μοντέλου

Dataset	Accuracy	F1 (macro)	Recall (macro)
Sentence Polarity (MR)	0.7281	0.7279	0.7281

Σχήμα 1: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – Baseline (mean pooling), MR

Ζητούμενο 11 – Κατηγοριοποίηση με Χρήση LLM

Prompts που χρησιμοποιήθηκαν:

Πίνακας 2: Αποτελέσματα ChatGPT

Dataset	**Accuracy**	**F1 (macro)**	**Recall (macro)**
Sentence Polarity	–	–	–
Semeval 2017	–	–	–

Επίδοση μοντέλου:

Ανάλυση αποτελεσμάτων:

Μέρος Β΄: Κύρια Άσκηση

Για τα ερωτήματα 1–8 χρησιμοποιήσαμε το **Sentence Polarity Dataset (MR)** για τα ερωτήματα 1–5 και και το **Semeval 2017 Task4-A** για τα ερωτήματα 6–7.

Ερώτημα 1 – Mean + Max Pooling

1.1 Υλοποίηση:

1.2 Διαφορές από την αρχική αναπαράσταση: Η νέα αναπαράσταση διαφέρει από την αρχική, επειδή η αρχική βασιζόταν αποκλειστικά στον μέσο όρο των embeddings μέσω mean pooling, ενώ τώρα η τελική αναπαράσταση προκύπτει από τη συνένωση (concatenation) του mean pooling και του max pooling. Οι δύο αυτές μορφές pooling παρέχουν διαφορετική αλλά συμπληρωματική πληροφορία, η οποία είναι χρήσιμη για το sentiment analysis των προτάσεων.

Το mean pooling υπολογίζει, για κάθε διάσταση του embedding, τη μέση τιμή της σε όλα τα tokens της πρότασης. Αν θεωρήσουμε ότι κάθε διάσταση του embedding αντιστοιχεί σε κάποια αφηρημένη σημασιολογική ιδιότητα, τότε το mean pooling δίνει μια συνολική εικόνα του νοήματος της πρότασης, σαν ένα είδος κέντρου βάρους. Έτσι αποτυπώνεται το γενικό περιεχόμενο και η μέση κατανομή των χαρακτηριστικών μέσα στην πρόταση. Ωστόσο, αυτή η διαδικασία τείνει να εξομαλύνει πολύ έντονες τοπικές ενδείξεις.

Αντίθετα, το max pooling κρατά, για κάθε διάσταση, τη μεγαλύτερη τιμή που εμφανίζεται σε κάποιο token της πρότασης. Με αυτόν τον τρόπο λειτουργεί σαν μηχανισμός εντοπισμού ισχυρών σημάτων: αν ακόμη και μία μόνο λέξη ενεργοποιεί πολύ έντονα μια συγκεκριμένη διάσταση, αυτή η πληροφορία διατηρείται στην τελική αναπαράσταση. Αυτό είναι ιδιαίτερα χρήσιμο στο sentiment analysis, όπου μία λέξη με έντονη συναισθηματική φόρτιση, θετική ή αρνητική, μπορεί να είναι καθοριστική για την τελική κλάση.

Συνεπώς, η συνένωση των δύο αναπαραστάσεων προσφέρει πλουσιότερη πληροφορία από την αρχική προσέγγιση, επειδή συνδυάζει τόσο τη συνολική, μέση πληροφορία της πρότασης όσο και την παρουσία έντονων χαρακτηριστικών σε κάθε διάσταση.

Πίνακας 3: Αποτελέσματα Ερωτήματος 1 (Mean+Max Pooling)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
Mean+Max Pooling	0.7462	0.7454	0.7462

Σχήμα 2: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – Mean+Max Pooling, MR

Ερώτημα 2 – LSTM

2.1 Validation set & Early Stopping: Για το validation split χρησιμοποιείται η συνάρτηση torch_train_val_split() στο main.py. Η συνάρτηση αυτή χωρίζει το αρχικό training set σε δύο μέρη: train_loader και val_loader, με default val_size=0.2, άρα χρησιμοποιείται το 20% του training set ως validation set, όπως φαίνεται και στην training.py. Στη συνέχεια εφαρμόζεται early stopping με την κλάση EarlyStopper στο main.py. Σε κάθε εποχή υπολογίζεται το val_loss, και αν αυτό επιδεινώνεται συνεχώς για 5 εποχές, η εκπαίδευση σταματά. Με αυτόν τον τρόπο το μοντέλο δεν εκπαιδεύεται για σταθερό αριθμό εποχών ανεξάρτητα από την απόδοσή του, αλλά σταματά όταν παύει να γενικεύει καλά στο validation set. Αυτό βοηθά στον περιορισμό του overfitting, αλλά και στη μείωση του υπολογιστικού κόστους κατά την εύρεση του βέλτιστου αριθμού εποχών εκπαίδευσης για το δεδομένο dataset.

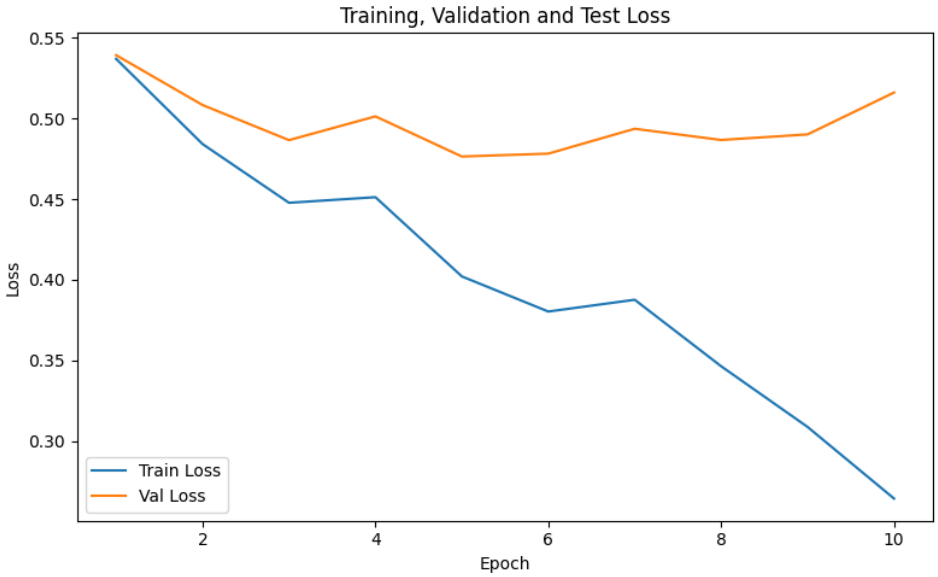
2.2 Χρήση τελευταίας εξόδου LSTM (h_N): Στο LSTM, κάθε λέξη δεν αναπαρίσταται μόνο από το embedding της, αλλά από μια νέα κρυφή κατάσταση h_i που ενσωματώνει και πληροφορία από τα προηγούμενα tokens. Σε αντίθεση με το απλό RNN, το LSTM διαθέτει μηχανισμό μνήμης μέσω του context/cell state c_t και τριών πυλών: forget, input και output gate. Με αυτές το δίκτυο μαθαίνει ποιες πληροφορίες πρέπει να διατηρήσει, ποιες να απορρίψει και ποιες να χρησιμοποιήσει στην τρέχουσα έξοδο. Έτσι μπορεί να μεταφέρει χρήσιμη πληροφορία σε μεγαλύτερες αποστάσεις και να αντιμετωπίζει καλύτερα το πρόβλημα των vanishing gradients. Για την αναπαράσταση της πρότασης χρησιμοποιείται η τελευταία πραγματική έξοδος h_N, δηλαδή η κρυφή κατάσταση που αντιστοιχεί στο τελευταίο έγκυρο token της πρότασης. Αυτό είναι σημαντικό γιατί τα zero-padded timesteps δεν ανήκουν στο πραγματικό κείμενο και δεν πρέπει να επηρεάζουν την αναπαράσταση. Στην υλοποίηση αυτό εξασφαλίζεται με χρήση των πραγματικών μηκών των προτάσεων και του pack_padded_sequence, ώστε το LSTM να επεξεργάζεται μόνο τα αληθινά tokens, και στη συνέχεια επιλέγεται το output στη θέση lengths - 1.

2.3 Αμφίδρομο LSTM (BiLSTM): Στο αμφίδρομο LSTM η πρόταση διαβάζεται και από τις δύο κατευθύνσεις: από αριστερά προς τα δεξιά και από δεξιά προς τα αριστερά. Έτσι η τελική αναπαράσταση κάθε θέσης περιέχει πληροφορία τόσο από το αριστερό όσο και από το δεξί συμφραζόμενο. Αυτό είναι ιδιαίτερα χρήσιμο στη γλώσσα, επειδή η ερμηνεία μιας λέξης συχνά εξαρτάται

όχι μόνο από ό,τι προηγήθηκε αλλά και από ό,τι ακολουθεί. Στην κατηγοριοποίηση προτάσεων, το bidirectional LSTM δίνει συνήθως πιο πλούσια αναπαράσταση του κειμένου από το μονοκατευθυντικό, επειδή συνδυάζει και τα δύο συμφραζόμενα πριν από την τελική ταξινόμηση. Στη δική μας υλοποίηση, με bidirectional=True, το μοντέλο στο dataset MR πέτυχε στο test set τα παρακάτω αποτελέσματα:

Πίνακας 4: Αποτελέσματα Ερωτήματος 2 (LSTM)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
LSTM (unidirectional)	0.7568	0.7546	0.7568



Σχήμα 3: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – LSTM, MR

Ερώτημα 3 – Self-Attention

3.1 Υλοποίηση & Αποτελέσματα (SimpleSelfAttentionModel): Στο SimpleSelfAttentionModel, κάθε λέξη προβάλλεται αρχικά στον χώρο των embeddings και στη συνέχεια, μέσω του μηχανισμού self-attention, παράγεται μια νέα αναπαράσταση για κάθε θέση, η οποία εξαρτάται όχι μόνο από το ίδιο το token αλλά και από τα υπόλοιπα tokens της πρότασης. Έπειτα εφαρμόζεται average-pooling στις τελικές αναπαραστάσεις των λέξεων, ώστε να εξαχθεί μία ενιαία αναπαράσταση για ολόκληρη την πρόταση πριν από το τελευταίο γραμμικό layer ταξινόμησης.

Η επίδοση του μοντέλου στο test set για το dataset MR φαίνεται στον παρακάτω πίνακα. Τα αποτελέσματα δείχνουν ότι τ ο self-attention μοντέλο μπορεί να αξιοποιήσει τις αλληλεπιδράσεις μεταξύ των λέξεων της πρότασης καλύτερα από ένα απλό bag-of-embeddings μοντέλο, αφού η αναπαράσταση κάθε λέξης εμπλουτίζεται με πληροφορία συμφραζομένων πριν από το τελικό pooling.

Πίνακας 5: Αποτελέσματα Ερωτήματος 3 (Self-Attention)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
Self-Attention	0.7492	0.7489	0.7492

Σχήμα 4: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – Self-Attention, MR

3.2 Queries, Keys, Values και Position Embeddings: Ο μηχανισμός attention μπορεί να ιδωθεί ως ένας σταθμισμένος μέσος όρος διανυσμάτων, όπου τα βάρη εξαρτώνται από το πόσο σχετικό είναι κάθε token με το token που εξετάζουμε κάθε φορά. Στην απλοποιημένη του μορφή, το attention output στη θέση i μπορεί να γραφεί ως

$$a_i = \sum_{j \leq i} \alpha_{ij} x_j$$

όπου τα α_{ij} είναι τα βάρη που καθορίζουν πόσο συμβάλλει η αναπαράσταση x_j στον υπολογισμό της νέας αναπαράστασης a_i.

Στην πλήρη μορφή του self-attention, κάθε είσοδος x_i παίζει τρεις διαφορετικούς ρόλους: ως query, ως key και ως value. Τα queries εκφράζουν τι πληροφορία 'αναζητά' το τρέχον token από τα υπόλοιπα tokens. Τα keys εκφράζουν με ποιον τρόπο κάθε token μπορεί να συγκριθεί με το τρέχον query, ενώ τα values είναι η πραγματική πληροφορία που θα χρησιμοποιηθεί στο τελικό σταθμισμένο άθροισμα.

Στην κλάση attention.Head, για κάθε είσοδο υπολογίζονται τρεις διαφορετικές γραμμικές προβολές:

$$q_i = x_i W_Q, \qquad k_i = x_i W_K, \qquad v_i = x_i W_V$$

Η ομοιότητα μεταξύ του token i και ενός άλλου token j υπολογίζεται από το εσωτερικό γινόμενο του query με το key, κλιμακωμένο με $\sqrt{d_k}$:

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

Στη συνέχεια, τα scores κανονικοποιούνται με softmax, ώστε να προκύψουν τα βάρη προσοχής:

$$\alpha_{ij} = \text{softmax}(\text{score}(x_i, x_j))$$

και το τελικό output της κεφαλής προκύπτει ως σταθμισμένο άθροισμα των value vectors:

$$\text{head}_i = \sum_{j \leq i} \alpha_{ij} v_j$$

Άρα, το attention output ενός token είναι μια νέα αναπαράσταση που συγκεντρώνει πληροφορία από τα άλλα tokens, δίνοντας μεγαλύτερο βάρος σε όσα θεωρούνται πιο σχετικά.

Τα position embeddings προστίθενται στα token embeddings, επειδή ο μηχανισμός attention από μόνος του δεν περιέχει πληροφορία για τη σειρά ή τη θέση των λέξεων. Χωρίς αυτά, το μοντέλο θα αντιμετώπιζε την πρόταση σαν ένα απλό σύνολο tokens. Με τα position embeddings, κάθε θέση της πρότασης αποκτά δική της διανυσματική πληροφορία, ώστε το μοντέλο να μπορεί να διακρίνει αν μια λέξη εμφανίζεται στην αρχή, στη μέση ή στο τέλος της πρότασης. Έτσι εισάγεται πληροφορία για τη σειρά των tokens, η οποία είναι κρίσιμη για τη σωστή κατανόηση του νοήματος.

Ερώτημα 4 – Multi-Head Attention

Στο ερώτημα αυτό επεκτείνουμε το απλό self-attention σε multi-head attention. Ενώ στο SimpleSelfAttentionModel υπάρχει μία μόνο κεφαλή προσοχής που υπολογίζει μία ενιαία σταθμισμένη σύνοψη των value vectors, στο MultiHeadAttentionModel χρησιμοποιούνται πολλές κεφαλές παράλληλα, καθεμία με δικά της βάρη προβολής για queries, keys και values. Έτσι κάθε κεφαλή μπορεί να εστιάζει σε διαφορετικές σχέσεις ή διαφορετικά πρότυπα μέσα στην πρόταση.

Πιο συγκεκριμένα, για κάθε κεφαλή c, οι είσοδοι προβάλλονται σε ξεχωριστά query, key και value διανύσματα, υπολογίζονται ανεξάρτητα τα βάρη προσοχής και παράγεται ένα επιμέρους head output. Στη συνέχεια, τα αποτελέσματα όλων των κεφαλών συνενώνονται και προβάλλονται ξανά στον αρχικό χώρο αναπαράστασης. Με αυτόν τον τρόπο το μοντέλο αποκτά πλουσιότερη και πιο εκφραστική αναπαράσταση της πρότασης σε σχέση με τη SimpleSelfAttention εκδοχή.

Για κάθε κεφαλή c, οι βασικοί υπολογισμοί περιγράφονται από τις εξισώσεις:

$$q_i^{(c)} = x_i W_Q^{(c)}, \qquad k_j^{(c)} = x_j W_K^{(c)}, \qquad v_j^{(c)} = x_j W_V^{(c)}$$

$$\alpha_{ij}^{(c)} = \text{softmax}\left(\frac{q_i^{(c)} \cdot k_j^{(c)}}{\sqrt{d_k}}\right)$$

$$\text{head}_i^{(c)} = \sum_j \alpha_{ij}^{(c)} v_j^{(c)}$$

και η τελική έξοδος του multi-head attention προκύπτει από τη συνένωση όλων των κεφαλών και την τελική γραμμική προβολή:

$$a_i = \left(\text{head}_i^{(1)} \oplus \text{head}_i^{(2)} \oplus \cdots \oplus \text{head}_i^{(A)}\right) W_O$$

Στη δική μας υλοποίηση, μετά την εφαρμογή του multi-head attention, χρησιμοποιείται average-pooling για την εξαγωγή της τελικής αναπαράστασης της πρότασης πριν από το τελευταίο γραμμικό layer ταξινόμησης. Η επίδοση του μοντέλου στο test set παρουσιάζεται στον αντίστοιχο πίνακα αποτελεσμάτων.

Πίνακας 6: Αποτελέσματα Ερωτήματος 4 (Multi-Head Attention)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
MultiHead-Attention (n_head=4)	0.7477	0.7475	0.7477
MultiHead-Attention (n_head=8)	0.7402	0.7401	0.7402

Σχήμα 5: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – Multi-Head Attention, MR

Ερώτημα 5 – Transformer-Encoder

Διαφορά από MultiHeadAttentionModel: Στο ερώτημα αυτό χρησιμοποιούμε έναν Transformer-Encoder για κατηγοριοποίηση συναισθήματος, διατηρώντας το average-pooling για την εξαγωγή της τελικής αναπαράστασης της πρότασης. Όπως και στα προηγούμενα μοντέλα, κάθε πρόταση μετατρέπεται αρχικά σε ακολουθία από token embeddings, στα οποία προστίθενται position embeddings, ώστε το μοντέλο να γνωρίζει τη θέση κάθε token μέσα στην πρόταση. Στη συνέχεια, η ακολουθία περνά από διαδοχικά Transformer blocks, όπου κάθε block αποτελείται από multi-head self-attention, feed-forward network, residual connections και layer normalization. Αν συμβολίσουμε την είσοδο ενός block με x, τότε η βασική του μορφή είναι:

$$x' = x + \text{MultiHeadAttention}(\text{LayerNorm}(x))$$

$$y = x' + \text{FeedForward}(\text{LayerNorm}(x'))$$

Μετά τη διέλευση από όλα τα blocks, εφαρμόζεται average-pooling στις τελικές αναπαραστάσεις των tokens, ώστε να προκύψει μία ενιαία αναπαράσταση πρότασης, η οποία οδηγείται στο τελικό γραμμικό layer ταξινόμησης.

Η βασική διαφορά του TransformerEncoderModel σε σχέση με το MultiHeadAttentionModel είναι ότι το δεύτερο χρησιμοποιεί ουσιαστικά ένα μόνο επίπεδο multi-head attention και ένα feed-forward layer, ενώ ο Transformer-Encoder στοιβάζει πολλά τέτοια blocks διαδοχικά. Άρα ο Transformer-Encoder έχει μεγαλύτερο βάθος και μπορεί να μάθει πιο σύνθετες και πιο αφηρημένες σχέσεις μεταξύ των λέξεων της πρότασης. Επιπλέον, στο τέλος εφαρμόζεται και τελικό layer normalization, το οποίο σταθεροποιεί περισσότερο την αναπαράσταση πριν από την ταξινόμηση.

Default τιμές παραμέτρων στην κλασική αρχιτεκτονική Transformer: Στα πειράματά μας μεταβάλαμε κυρίως τον αριθμό των κεφαλών attention (n_head) και τον αριθμό των encoder layers (n_layer). Στην κλασική αρχιτεκτονική του Transformer των ἄσωανι et al., οι βασικές default τιμές για τον encoder είναι:

- n_head = 8
- n_layer = 6

- model dimension $d = 512$
- feed-forward dimension $= 2048$
- $d_k = d_v = 64$

Στη δική μας υλοποίηση, επειδή χρησιμοποιούμε προεκπαιδευμένα embeddings διαστάσεων 200, το model dimension καθορίζεται από αυτά και δεν είναι 512. Η επίδοση του μοντέλου στο test set για το dataset MR παρουσιάζεται στον παρακάτω πίνακα αποτελεσμάτων. Παρατηρούμε ότι η καλύτερη επίδοση επιτεύχθηκε με n_head = 4 και n_layer = 6, γεγονός που δείχνει ότι το μεγαλύτερο βάθος βοήθησε το μοντέλο να μάθει πιο εκφραστικές αναπαραστάσεις για την ταξινόμηση συναισθήματος.

Πίνακας 7: Αποτελέσματα Ερωτήματος 5 (Transformer-Encoder)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
Transformer-Encoder (n_head=4, n_layer=3)	0.7492	0.7462	0.7492
Transformer-Encoder (n_head=8, n_layer=6)	0.7477	0.7456	0.7477
Transformer-Encoder (n_head=4, n_layer=6)	0.7704	0.7699	0.7704

Σχήμα 6: Καμπύλες εκπαίδευσης (training & test loss) ανά εποχή – Transformer-Encoder, MR

Ερώτημα 6 – Pre-Trained Transformers (Zero-Shot)

Πίνακας 8: Αποτελέσματα Pre-Trained Transformers – Sentence Polarity (MR)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
siebert/sentiment-roberta-large-english	0.9260	0.9260	0.9260
textattack/bert-base-uncased-SST-2	0.8988	0.8987	0.8988
distilbert-base-uncased-finetuned-sst-2-english	0.8912	0.8912	0.8912
aychang/roberta-base-imdb	0.8489	0.8485	0.8489
lvwerra/distilbert-imdb	0.8202	0.8195	0.8202

Πίνακας 9: Αποτελέσματα Pre-Trained Transformers – Semeval 2017

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
cardiffnlp/twitter-roberta-base-sentiment	0.7238	0.7222	0.7229
cardiffnlp/twitter-roberta-base-sentiment-latest	0.7205	0.7226	0.7330
finiteautomata/bertweet-base-sentiment-analysis	0.7178	0.7181	0.7302
cardiffnlp/twitter-xlm-roberta-base-sentiment	0.6926	0.6886	0.6929
Seethal/sentiment_analysis_generic_dataset	0.6184	0.6093	0.6181

Σχολιασμός αποτελεσμάτων:

Ερώτημα 7 – Fine-Tuning Pre-Trained Transformers

Πίνακας 10: Αποτελέσματα Fine-Tuned Transformers – Sentence Polarity

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**

Πίνακας 11: Αποτελέσματα Fine-Tuned Transformers – Semeval 2017

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**

Σχολιασμός αποτελεσμάτων:

Ερώτημα 8 – Διάλογος με ChatGPT για GPT Κώδικα

Παραδείγματα διαλόγου:

Σχολιασμός απόδοσης:

Συγκεντρωτικά Αποτελέσματα

Πίνακας 12: Σύγκριση όλων των μοντέλων – **Sentence Polarity (MR)** (εκτός Pre-Trained Semeval)

Μοντέλο	**Accuracy**	**F1 (macro)**	**Recall (macro)**
Baseline DNN (mean pooling)	0.7281	0.7279	0.7281
Mean + Max Pooling	0.7462	0.7454	0.7462
LSTM	0.7568	0.7546	0.7568
Self-Attention	0.7492	0.7489	0.7492
Multi-Head Attention (n_head=4)	0.7477	0.7475	0.7477
Transformer-Encoder (n_head=4, n_layer=6)	0.7704	0.7699	0.7704
Pre-Trained best (siebert, MR)	0.9260	0.9260	0.9260
Pre-Trained best (cardiffnlp, Semeval)	0.7238	0.7222	0.7229
Fine-Tuned (best)	–	–	–